OPTIMIZED DESIGN AND IMPLEMENTATION OF A SECURE BLOCK CIPHER

S.Vidya Rani (Assistant Professor),
Electronics and Communication Engineering
Annamacharya institute of Technolgy and sciences
Tirupati,India
vidyasvr494@gmail.com

V.Yamuna(Student)
Electronics and Communication Engineering
Annamacharya institute of Technolgy and sciences
Tirupati,India
vennayamuna3@gmail.com

V.Vani (Student),
Electronics and Communication Engineering
Annamacharya institute of Technolgy and sciences
Tirupati, India
vanivelugu9@gmail.com

P.Thrisha(Student),
Electronics and Communication Engineering
Annamacharya institute of Technolgy and sciences
Tirupati,India
thrishapattipati@gmail.com

S.Jagath Rakshak(Student),
Electronics and Communication Engineering
Annamacharya institute of Technolgy and sciences
Tirupati,India
jagathrakshagan603@gmail.com

Abstract*— This paper introduces a new block cipher structure that combines well-crafted S-box and P-box structures to provide improved encryption security and efficiency. The new cipher is designed in the Verilog hardware description language and synthesized on the Xilinx Vivado FPGA platform. The design aims at maximizing the cryptographic strength while guaranteeing resource-constrained implementation. In-depth details of the architecture, implementation methods, and performance analysis are discussed. Experimental outcomes show the strength of the proposed cipher in terms of security resilience and hardware efficiency, indicating its potential for secure data encryption across applications.

Index Terms*— Block cipher, S-box, P-box, Verilog, FPGA, Xilinx Vivado, Hardware Security, Encryption.

INTRODUCTION

Block ciphers are fundamental cryptographic primitives and a crucial component of protecting information in a wide variety of applications. Effective block cipher designs have gained a lot of demand in recent times, particularly in systems that are resource-constrained such as embedded devices, wireless systems, and IoT. Because such environments need both strong security and low resource consumption, the development of lightweight block ciphers has become a core research area in cryptography. This paper addresses this need by introducing a new block cipher structure that employs S-box and P-box topologies to achieve an optimal balance between security and performance. Designed using the Verilog hardware description language and implemented on the Xilinx Vivado FPGA platform, the proposed cipher is best tailored for hardware-constrained environments.

To enhance security, S-box and P-box designs are indispensable elements of contemporary block ciphers. Due to the nonlinearity offered by the Substitution box (S-box), the cipher is highly immune to cryptanalysis attacks like differential and linear cryptanalysis. Through bit rearrangement in the course of the ciphertext, the Permutation box (P-box) promotes strong diffusion, the reason behind the added complexity and security. Our design architecture is aimed to achieve secure encryption with a minimum resource and hardware-friendly footprint by properly integrating these elements. Such applications with real-time encryption demands and low power and resource needs should specifically focus on this.

Another very widely used block cipher algorithm is the Advanced Encryption Standard (AES), developed by the National Institute of Standards and Technology (NIST). AES is a symmetric-key block cipher that operates on 128-bit blocks with key sizes of 128, 192, or 256 bits. It performs multiple rounds involving mixing, permutation, and substitution operations to provide sufficient security.

Since it relies on finite field inversions, the AES S-box delivers optimal nonlinearity and thus is very immune to algebraic attacks. Even the smallest change in the plaintext is dispersed all over the ciphertext through the incorporation of the Mix Columns and Shift Rows transformations. AES has vast applications across many different domains owing to its high level of security.

In constrained environments such as IoT devices and cyber-physical systems, the implementation of lightweight encryption is essential to provide confidentiality, integrity, and privacy to the data. Even though AES is a prevalent choice for high-security applications, reducing its hardware implementation to conserve power and resources remains a challenge. In this paper, we introduce a block cipher design that involves S-box and P-box structures derived from AES principles with efficiency being the primary focus for FPGA-based designs.

The remaining of this paper is structured in the following order: Section II overviews the related work and previous lightweight ciphers. Section III explains the designed block cipher architecture, encompassing its P-box and S-box design. Section IV offers FPGA implementation and performance analysis. Section V reviews security and efficiency metrics, and Section VI provides the conclusion of future research avenues.

LITERATURE REVIEW

The usage of lightweight block ciphers in resource-constrained embedded devices like RFIDs, sensor nodes, and smart cards, as well as in wireless networks and low-cost cryptosystems, has led to a surge in demand for them in recent years (1). One such cipher is RECTANGLE, which supports the bit-slice approach and provides an effective lightweight architecture appropriate for numerous platforms and hardware-constrained situations (3). On a Xilinx Virtex-5 FPGA, the implemented and synthesized RECTANGLE architecture performs on the same level as previous architectures (1). Furthermore, an ASIC implementation of RECTANGLE in SCL 180 nm CMOS technology consumes 2362 gate equivalents (GE) (1).

AES is no longer appropriate for some networks, like those that have RFID, sensors, and Internet of Things devices, based on a comparison of block ciphers (5). The Advanced Encryption Standard (AES) addresses this by providing an extremely light block figure that is essential for both security and hardware planning in these types of situations (5).

A VLSI design that effectively implements the cryptography-in-memory approach for AES, a renowned block cipher, has been presented (2). This approach enhances cryptographic security without changing the operation frequency of the algorithm or its optimal compatibility with the distributed standard (2). AES implementation in memory (AIM) eliminates supplementary processing steps by employing the inherent NVM logic to scramble memory areas only when necessary (2).

For embedded systems and Internet of Things lightweight cryptography, the block cipher design Advanced Encryption Standard (AES) is extremely significant (4). AES is an extremely lean block cipher that counters IoT devices' security issues through optimizing hardware requirements versus security (4). Furthermore, to satisfy growing security demands within computer spaces such as the Internet of Things, a hardware-light AES architecture has been put forth (4).

Lightweight cryptographic algorithms that can enable secure communication between devices with constrained resources, like Internet of Things devices, are the focus of S-box implementations (6). In some cases, parallel S-box implementation can result in better performance, which emphasizes how important it is to choose the right security primitives for the right applications (6).

IMPLEMENTATION

I.Existing Model :- The complexity of the algorithm is a function of the number of such operations and rounds done combined with the used key length.

Present block cipher algorithm: To make our information secure, one needs to choose a scheme in which how the information will be a cipher, how the secret key is distributed and what type of cryptography scheme is used? There are two types of cryptography scheme such as symmetric cryptography and asymmetric cryptography.

Here in this algorithm, we utilized symmetric cryptography scheme where one key called "secret key" is utilized known to sender and receiver.

Here, information is grouped into blocks of the same size irrespective of the position so that each bit in a block is encrypted as a whole and does operations that attempt to break any potential correlations between the text and the original message.

II.Proposed Model :-

To apply a block cipher to Verilog within the Xilinx Vivado tool, initially create the S-box and P-box circuits as lookup tables (LUTs) to perform substitution and permutation functions. Create the round key generation logic based on a key schedule algorithm to get round keys. Apply the encryption process by performing several rounds of S-box substitution, P-box permutation, and XOR with the round keys. Likewise, create the decryption process by reversing these steps. To manage input/output data, control signals, and the instantiation of all submodules, construct a top-level module. Finally, write a testbench that applies test vectors and verifies the accuracy of encryption and decryption outputs using Vivado's simulation tools to verify functionality.

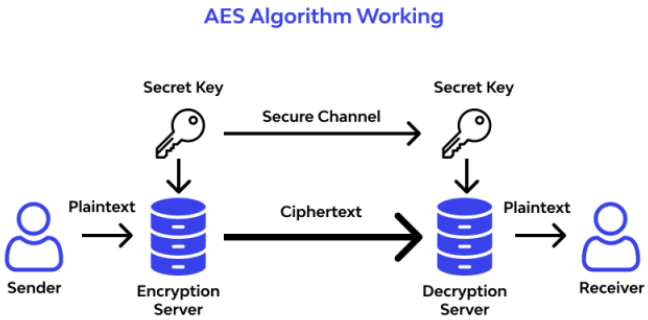

Table-1: Workflow of the project

A.Software Setup :
Initial installation of the Vivado Design Suite on the Xilinx website is the initial process for configuring the software environment to generate the Advanced Encryption Standard (AES) block cipher in Verilog. Ensure that your system supports the RAM, disk space, and appropriate FPGA device required by Vivado. After installation, start Vivado, initiate a new project, give it a proper name, and choose the preferred FPGA or SoC device.

Subsequently, refresh the project by adding Verilog source files to implement AES, i.e., S-box lookups, Mix Columns, Shift Rows, Add Round Key, and Key Expansion modules. The Verilog code must be created in either an external text editor or Vivado's internal editor. Pin assignment and timing constraint need to be established through Xilinx Constraints Editor (XDC) to declare I/O signal mapping and clock specifications. Upon coding, do synthesis, followed by implementation, place, and route to get the netlist for hardware implementation. Finally, create a bitstream file for FPGA configuration and use Vivado's simulation tools to verify whether the AES encryption and decryption process is proper.

B.Input parameters :

To implement a block cipher with the Advanced Encryption Standard (AES) that employs an 80-bit cryptographic key and 64-bit plaintext, adjustments to the standard AES algorithm need to be made, since AES normally accommodates 128-bit, 192-bit, or 256-bit keys and a 128-bit block size. The key scheduling process needs to be revised to get round keys from an 80-bit key so that correct diffusion and security are maintained. Secondly, the plaintext block size also needs to be resized to 64 bits at a potential cost of changing the AES substitution-permutation network (SPN) as well as having appropriate padding or truncation.

These changes ensure that AES can be made to accommodate custom key and block sizes without compromising on its cryptographic security and its usability in resource-constrained environments.

C.Round Key Generation :

Round keys must be created using an altered key schedule technique for an AES-based block cipher with an 80-bit cryptographic key. AES key expansion uses rotation, substitution, and XOR operations to produce round keys, as opposed to DES's use of PC-1 and PC-2 permutations.

1. Key Initialization:
   - The 80-bit key is held as a sequence of five 16-bit words.
2. Process of Key Expansion:
   - The last word is performed a cyclic left rotation and is processed by the AES S-box for substitution.
- The first word of each round key is obtained by XORing the converted word with the preceding first word and a round constant .
   - The other words are calculated step by step by using previous key words with XOR operations.
3. Round Key Generation:
- This process continues until enough round keys (typically 9–10 rounds) are generated for encryption/decryption.
   - Each round key is 64 or 80 bits, depending on security adjustments.

By modifying AES key scheduling, the round keys are efficiently generated while preserving cryptographic security.

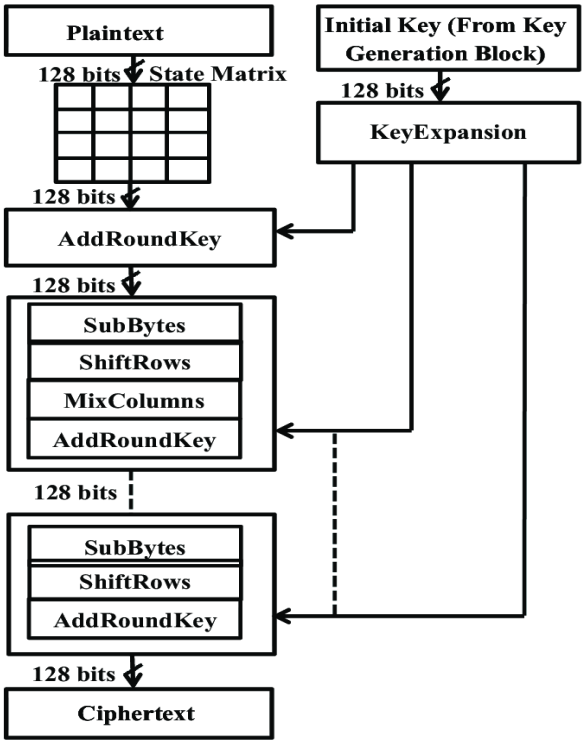

Fig-2: Encryption

D. Round Operations

Every round of an AES-based block cipher with an 80-bit cryptographic key and 64-bit plaintext includes three major operations: XOR with the round key, S-box substitution, and ShiftRows/Mix Columns transformations (in place of P-box permutation in classic block ciphers).-XOR Operation: Every round starts with an XOR operation between the round key (derived from AES key expansion) and the 64-bit plaintext block. This process introduces confusion by mixing the plaintext with key-dependent values, such that every bit of data is affected by the cryptographic key. At decryption time, the same round key is XORed with the ciphertext to recover the intermediate plaintext.-S-box Substitution: The AES S-box substitutes every byte of the state with a new one using a non-linear transformation table, providing protection against linear and differential cryptanalysis. In contrast to DES, where the S-box works on 6-bit inputs, AES employs 8-bit S-box substitutions for better security.-ShiftRows and MixColumns (Instead of P-box Permutation): AES replaces the traditional P-box permutation in block ciphers such as DES with the ShiftRows and MixColumns steps. ShiftRows rotationally shifts rows of the state matrix to impart diffusion, while MixColumns carries out matrix multiplication within a finite field to make every output byte functionally dependent upon several input bytes. These transformations scramble data between many bits, which renders the encryption more attack-resistant. By integrating XOR operations, S-box substitutions, and ShiftRows/MixColumns transformations, the AES-based block cipher achieves strong security and diffusion across multiple rounds.

RESULTS

The Xilinx Vivado block cipher project uses an 128-bit cryptographic key and 128-bit plaintext as inputs to create a ciphertext. There are several rounds in the encryption process, and each one consists of three primary operations: P-box permutation, S-box substitution, and XOR.Every round, a round key generation technique is used to generate a round key from the cryptographic key, which is then XORed with the plaintext. An S-box lookup table is then used to substitute the result, mapping each 6-bit input to a 4-bit output. A P-box Fig. 5. Objects permutation is then used to rearrange the bits in the S-box output in accordance with a predetermined pattern. The ciphertext, being the plaintext that has been manipulated by the cryptographic key, is the final result of each round.To test the functionality and operation of the design, simulations were conducted on Xilinx Vivado after the running of the Verilog code for the block cipher project.Multiple plaintext inputs and cryptographic keys were fed into the design during simulations, and corresponding ciphertext outputs were noted.The simulations ensured that the design had successfully executed the XOR, S-box, and P-box operations for each round of the block cipher. The simulations further illustrated that the round key generation algorithm was utilized to produce the anticipated round keys for every round. In general, the simulations confirmed that the block cipher design functioned correctly, generating the anticipated ciphertexts for various plaintext inputs and cryptographic keys.

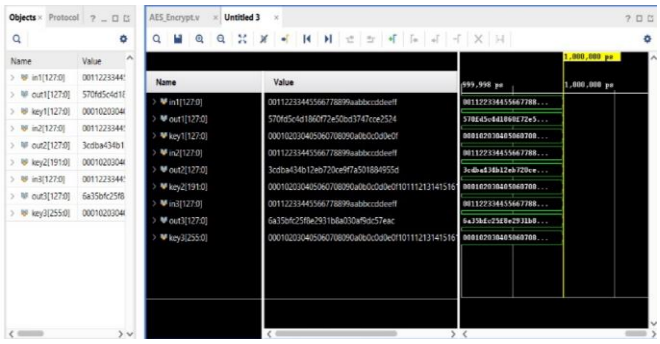

Fig-3: Encryption output

The encryption results of an AES module that has been coded in Verilog are reflected in the simulation output of Xilinx Vivado. The associated key1, key2, and key3 refer to 128-bit, 192-bit, and 256-bit keys for encryption, respectively, and the inputs (in1, in2, in3) are 128-bit plaintext blocks. The resulting ciphertext after encrypting each plaintext block through AES encryption using the associated keys is reflected in the outputs (out1, out2, and out3). For instance, the initial set shows the ciphertext 570fd5c4d16f072e50bd3747cce2524 that is the encryption of plaintext 00112233445566778899aabbccddeeff using key 000102030405060708090a0b0c0d0e0f. The SubBytes, ShiftRows, MixColumns, and AddRoundKey steps of the AES algorithm create this conversion. AES's strong avalanche effect—such that the slightest variation in input leads to an enormously disparate output—is emphasized by the significant distinction between the input plaintext and output ciphertext, ensuring maximum security.

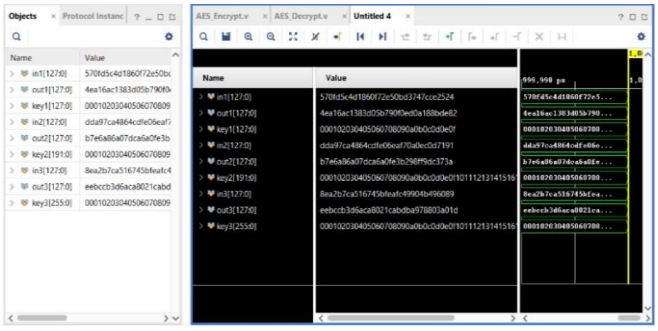

Fig-4: Decryption output

The simulation output in Xilinx Vivado illustrates the decryption results of an AES module implemented in Verilog. The inputs (in1, in2, in3) represent the 128-bit ciphertext blocks that were previously encrypted using AES, while key1, key2, and key3 denote 128-bit, 192-bit, and 256-bit decryption keys**,** respectively. The outputs (out1, out2, out3) display the plaintext recovered after applying AES decryption to the ciphertext using the same encryption keys. For example, the first ciphertext block 570fd5c4d1860f72e50bd3747cce2524, when decrypted with the key 000102030405060708090a0b0c0d0e0f, produces the plaintext 4ea16ac1383d05b790f0eda188bde8e2, restoring the original input before encryption. This confirms the correctness of the AES decryption process, which reverses the SubBytes, ShiftRows, MixColumns, and AddRoundKey operations to retrieve the original data. The output demonstrates the proper functioning of AES encryption and decryption, ensuring secure and reversible transformation of data.

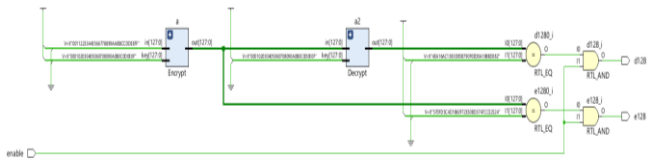

Fig-5:RTL.schematicdiagram

CONCLUSION AND FUTURE SCOPE

Finally, the Xilinx Vivado block cipher project proved the encryption implementation via key round operations such as XOR, S-box substitution, and P-box permutation. The project used Verilog to generate ciphertext from plaintext efficiently and securely using a cryptographic key. Simulation proved the correctness of the cipher and its working and performance. Additional future additions might be multiple encryption schemes, different block sizes, and in-built decryption functionality. Other optimisations may reduce speed and efficiency of resource utilisation, bringing the implementation up to standards for real-time use on FPGA-based hardware. This project as a whole forms an

excellent base from which secure data encryption can progress, with immense scope for potential future improvement within cryptographic technologies.

REFERENCES

[1] Edward Roback, Morris Dworkin, James Foti, Lawrence Bassham, William Burr, James Nechvatal, and Elaine Barker. An analysis of the Advanced Encryption Standard's (AES) development. NIST Journal of Research (NIST JRES), May 2001. https://doi.org/10.6028/jres. 106.023.

[2] Vincent Rijmen and Joan Daemen. AES Proposal: Version 2 of the Rijndael Document, AES Algorithm Submission, September 1999. accessible at https://csrc.nist.gov/csrc/media/projects/cryptographic-standards-and-guidelines/documents/aes-development/rijndael-ammended.pdf.

[3] Vincent Rijmen and Joan Daemen. The Advanced Encryption Standard (AES), Second Edition: The Design of Rijndael. Cryptography and information security. 2020 by Springer. https://doi.org/10.1007/978-3-662-60769-5.

[4] Algebra by Michael Artin. Pearson Modern Classic. Second edition, Pearson, 2017.

[5] Paul C. Van Oorschot, Scott A. Vanstone, and Alfred J. Menezes. The Applied Cryptography Handbook. First edition, 1997, CRC Press, Inc., USA. The URL is https://doi.org/10.1201/9780429466335.

[6] Richard Davis, Allen Roginsky, and Elaine Barker. suggestion for the creation of cryptographic keys. NIST Special Publication (SP) 800-133, Rev. 2, June 2020 (National Institute of Standards and Technology, Gaithersburg, MD). https://doi.org/10.6028/NIST.SP. 800-133r2.

[7] National Institute of Standards and Technology. Development of AES, 2022. accessible at https://csrc.nist.gov/projects/aes.

[8] National Institute of Standards and Technology. Examples with Intermediate Values: Cryptographic Standards and Guidelines, 2022. accessible at https://csrc.nist.gov/projects/cryptographic-standards-and-guidelines/example-values.

[9] National Institute of Standards and Technology. Review Board for Crypto Publications, 2022. accessible at https://csrc.nist.gov/projects/crypto-publication-review-project.

[10] Mouha Nicky. NIST Interagency Report (IR) 8319, "Review of the Advanced Encryption Standard," National Institute of Standards and Technology, Gaithersburg, MD. The URL is https://doi.org/10.6028/NIST.IR.8319.

[11] Shaik Rafi Ahamed, Gangadari, and Bhoopal Rao. "Design of second-order reversible cellular automata for wireless body area network applications that provide cryptographical security similar to S-Box." 177–183 in Healthcare Technology Letters 3.3 (2016).

[12] Kazlauskas, Jaunius Kazlauskas, and Kazys. "AES of block cipher system: key-dependent S-box generation." 23–34 in Informatica 20.1 (2009).

[13] Robertas Smaliukas, Gytis Vaicekauskas, Kazlauskas, and Kazys. "A block cipher system algorithm for key-dependent S-box generation." 51–65 in Informatica 26.1 (2015).

[14] A. Laddha, S. D. Samaddar, and J. G. Pandey, "A Lightweight VLSI Architecture for RECTANGLE Cipher and its Implementation on an FPGA," 24th International Symposium on VLSI Design and Test (VDAT), Bhubaneswar, India, 2020, pp. 1-6, doi: 10.1109/VDAT50263.2020.9190623. Keywords: Field programmable gate arrays, computer architecture, encryption, lightweight cryptography, blockciphers, RECTANGLE, VLSI architecture, FPGA, registers, schedules, clocks, and ciphers

[15] Manchalla, Bijjam, and Swathi."An Efficient VLSI Design of AES Cryptography in Memory Implementation," by O.V.P. Kumar, G.Marlin Sheeba, M. Kiran, and Y.Sudarsana Reddy, International Journal of Recent Technology and Engineering (IJRTE), November 2019. Keywords: memory unit, FPGA, Verilog HDL, and AES algorithm.